PES University — Dept. of CSE

Lab 3: Architectural Pattern Justification

Problem Statement #07: Hostel Maintenance & Issue Ticketing System

Name: Aakash Agarwal | SRN: PES2UG24CS009 | Section: A (CSE)

Architecture Selection: We chose Layered Architecture for the Hostel Maintenance & Issue Ticketing System.

Reason 1 — Requirements map cleanly onto layers

The system's functional requirements naturally separate into three concerns: resident/warden-facing interaction (FR-001, FR-002), ticket lifecycle and routing logic (FR-003 to FR-005), and persistent storage of ticket records and photo attachments. A Layered Architecture (Presentation, Business, Data) maps directly onto these boundaries without introducing the distributed-system overhead that a hostel-scale application does not need.

Reason 2 — Matches the system's scale and team size

Hostel maintenance ticketing serves a single hostel's residents and staff, a predictable and moderate load rather than the elastic, high-scale traffic that justifies microservices. A layered design is simpler to develop, test, and deploy, and it keeps the SLA-escalation logic (NFR-001) centralized in one Business layer instead of coordinated across independently deployed services.

Security Advantage: Confining data access behind the Data layer's Ticket & Media Repository Component means the Presentation layer never touches the database directly; every request is funneled through the Business layer first. This single, auditable access boundary enforces the role-based access control and the at-rest/in-transit encryption required by NFR-002.

Performance Benefit: Because the SLA Monitor and Ticket Management components sit together in the same Business layer, the 24-hour SLA escalation check (NFR-001) runs as an in-process call rather than a network call, so escalation alerts fire with minimal latency instead of incurring the round-trip overhead a microservices split would add between a ticket-status change and its escalation check.

Hostel Maintenance & Issue Ticketing System — Layered Architecture (Component Diagram)

